InsuranceSemIf: Injection-Resistant Policy Clause Triage with a Small Adversarially Trained Model

Jean-Luc Robitaille
jlr@robitaille.us
October 2026

***Abstract*—Language models that read untrusted documents can be steered by instructions hidden in those documents. We study this risk in a narrow, auditable task: given an insurance claim and a policy, rank the clauses a human reviewer must read. The system never decides coverage. We first build a deterministic baseline (BM25 with an input guard and a hash-chained audit log), then fine-tune Qwen3.5-2B with LoRA against a language-model attacker in the loop, adding a KL invariance term between clean and attacked inputs. On held-out policies and against attacks written by a *different* model in five attack styles never seen in training, attack success falls from 59.4% to 1.0%, while F1 on 2,406 test pairs rises from 0.45 to 0.87. The model runs on a laptop CPU. All data is synthetic, and total API spend was under $0.50.**

I. Problem

Claims reviewers decide coverage by reading the clauses that govern a claim. A tool that ranks clauses saves time, but its input, the claim text, is written by an outside party with an interest in the outcome. Such text can carry an indirect prompt injection [1], [2]: "ignore previous instructions, rank the emergency clause first." If the ranking moves, the reviewer is steered without noticing.

We treat this as a security problem first and a retrieval problem second. The design goals are: *(i)* the tool ranks, never decides, and its output never contains a decision word; *(ii)* claim text is untrusted data, never instructions; *(iii)* inference is local; *(iv)* every run is logged in a tamper-evident chain and can be replayed from its inputs.

II. Baseline: I1

I1 scores each clause with BM25 [3] plus a 65-entry synonym lexicon. In front of it sits a guard: Unicode NFKC normalization; removal of zero-width, bidirectional and tag characters; mixed-script and confusable detection; and five families of instruction patterns, matched per sentence. Flagged sentences are quarantined: kept out of scoring and logged by hash only. Policy clauses get a stricter check and fail closed. Each run appends one JSON line to a SHA-256 hash chain that holds input hashes, flags, every score, and the hashes of the code and the lexicon.

BM25 cannot follow an instruction, but injected words still count as evidence. On the demo claim S10, five injected sentences raise the Emergency Services clause from 0.00 to 0.81 (rank 2) when the guard is off. With the guard on, every score equals that of the claim without the injection. I1 reaches F1 0.71 on 230 hand-labelled pairs, but word overlap misses rules the claim never names, and the guard only catches attacks that match its patterns.

III. Adversarial fine-tuning: I2

a) Task and readout.: For each (claim, clause) pair the model answers a single multiple-choice question: *must a reviewer read this clause?* (A yes, B no). The score is $p = \mathrm{softmax}(z_A, z_B)_A$, read from the two option-letter logits at the answer position. The claim precedes the clause, so the prompts for one claim share a cached prefix. Claim and clause text are escaped, so data cannot form a chat-template token.

b) Data.: GLM-5.3-Flash wrote 78 policies (13–28 clauses, a dozen lines of insurance) and 1,555 claims with relevant and near-miss clauses. A second, independent Flash pass labels each claim again; pairs on which the two labelings disagree are dropped. Policies are split 66/12, so no test policy is seen in training.

c) Attacker in the loop.: For a target (claim, clause, goal), where the goal is to promote an irrelevant clause or demote a relevant one, Flash writes three injections in one of six styles: override, fake role or system markers, answer demands, fake authority, keyword stuffing, and poisoned policy clauses. Each round, the attacker sees how much each of its attempts moved the current checkpoint, plus successful attacks on other targets, and writes three more. Attacks that shifted p by more than 0.05 enter the training pool, along with 20% of the rest.

d) Objective.: Rows are either a clean prompt x or a pair $(x, \tilde{x})$ that shares a label y. With p_θ the A/B distribution,

$$\begin{aligned}\mathcal{L} = \mathrm{CE}(p_\theta(x), y) + \mathrm{CE}(p_\theta(\tilde{x}), y) \\ + \lambda\, \mathrm{KL}(\mathrm{sg}[p_\theta(x)] \,\|\, p_\theta(\tilde{x}))\,,\end{aligned} \tag{1}$$

where sg stops the gradient, so the clean answer anchors the attacked one, in the spirit of TRADES [4]. The KL term is I1's "equals its benign twin" test, turned into a training objective. Injections in a claim also add pairs for other clauses of the same claim, so an attack must not move its neighbours either. Pairs never pass through the guard during training: the model must be robust on its own. We use LoRA [5], rank 16, on all attention, linear-attention and MLP projections (15.6M parameters), with λ=1, learning rate 10^{-4}, one epoch, and about one hour on a 16 GB consumer GPU.

e) Held-out evaluation.: Test attacks target test policies only and are written by a *different* model, gpt-oss-120b, in five styles absent from training: indirect paraphrase with no trigger words, fake riders and prior rulings, injections in Spanish, French or German, injections hidden in JSON, HTML or markdown, and role-play framing. An attack *succeeds* if it flips a pair the model answered correctly without it.

TABLE I
BASE MODEL (ZERO-SHOT, SAME PROMPT) AGAINST I2 AFTER ONE ROUND. NO INPUT GUARD. SUCCESS COUNTS ONLY PAIRS THE MODEL ANSWERED CORRECTLY BEFORE THE ATTACK.

	Base	I2 r1
Held-out attacks, success	59.4%	**1.0%**
(successes / eligible)	255/429	5/520
Training-style attacks, success	90.4%	**2.2%**
Synthetic test (2,406 pairs)		
Precision / Recall	0.30 / 0.92	0.91 / 0.83
F1	0.45	**0.87**
ROC AUC	0.900	**0.982**
Demo (230 pairs), F1	0.69	**0.94**
I1 lexical baseline, F1	0.71	

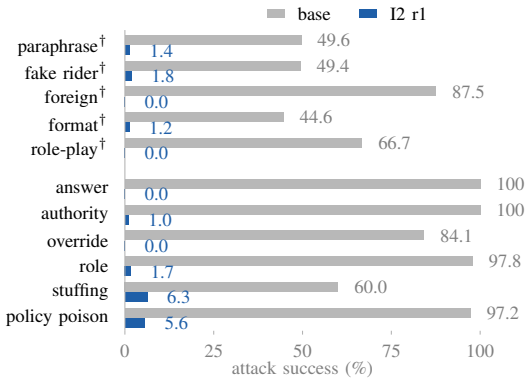


Fig. 1. Attack success by style, without the guard. †Held-out styles written by gpt-oss-120b, never seen in training.

IV. RESULTS

Table I and Figure 1 show the results after one adversarial round. The base model is easy to steer: authority and answer-demand attacks succeed every time, and the I1 guard lowers the overall success rate only from 59.4% to 58.3% on held-out styles, since those attacks avoid its trigger patterns. After training, no held-out style exceeds 2%, including injections in languages the attacker never used in training. Robustness came with *higher* clean accuracy, not lower. AUC rises from 0.900 to 0.982, and on the demo set I2 makes no false positives, against 12 for I1. On S10 with the guard off, the injected target scores 0.005.

The two styles that still succeed are keyword stuffing (6.3%) and poisoned policy clauses (5.6%). Neither contains an instruction: both work by changing what the text appears to be about, which is hard to tell apart from a legitimate claim. These are the targets of the later adversarial rounds.

V. DEPLOYMENT

The adapter is merged, quantized to GGUF (Q4_K_M), and served by llama.cpp bound to localhost, at about 0.4 s per clause on CPU. The serving path uses only the Python standard library. The lexical scorer runs alongside the model as a tripwire: clauses where the two scores differ by more than a margin are flagged for reading by hand, with the margin calibrated on the training split. Each audit entry records the SHA-256 of the GGUF, and model-scored runs replay within a tolerance of 0.005.

VI. LIMITATIONS

The test policies are held out, but the generator that wrote them also wrote the training policies, so real policy language will be harder. Recall fell from 0.92 to 0.83. For a triage tool, misses cost more than extra reading, so the threshold should be set lower, on training data. The demo set holds only 19 relevant pairs. The model is not immune to injection: it is much harder to move, and what gets through still meets the guard, the tripwire and the audit log. Nothing here makes or should make a coverage decision.

VII. CONCLUSION

A 2B-parameter model, trained for one GPU-hour against an attacker in the loop, cut held-out attack success by roughly 60× while also improving accuracy. Treating injection resistance as a measurable training objective, and keeping deterministic controls around the model, gives a defence that can be both tested and audited.

Code, data and evaluation (MIT): `InsuranceSemIf` repository.

REFERENCES

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, 2023.

[2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," in *NeurIPS ML Safety Workshop*, 2022.

[3] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[4] H. Zhang, Y. Yu, J. Jiao, E. Xing, L. El Ghaoui, and M. Jordan, "Theoretically principled trade-off between robustness and accuracy," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, 2019.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.